Python Developer – Django (Fresher)

Company: Nexgensis Technologies

Python & OOPs

1. What are the key features of Python, and why is Python suitable for backend development?
2. Explain the difference between **list, tuple, set, and dictionary** with use cases.
3. What is the difference between **mutable and immutable** objects in Python?
4. What is the difference between `==` and `is` in Python?
5. What are `*args` and `**kwargs`? Where would you use them?
6. How does exception handling work in Python? Explain `try`, `except`, `else`, and `finally`.
7. What is a **class and object** in Python? Give an example from a real application.
8. Explain the four pillars of OOPs with Python examples.
9. What is inheritance in Python? Explain with an example.
10. What is method overriding, and how is it implemented in Python?
11. What is the purpose of the `__init__()` method?
12. What is `self` in Python, and why is it required inside class methods?

Django & MVT

13. What is Django, and why would you choose Django for developing a web application?
14. Explain Django's **MVT architecture** with a practical example.
15. What is the difference between a **Django project and a Django app**?
16. Explain the complete flow when a user sends a request to a Django application.
17. What is the purpose of `urls.py` in Django?
18. What is a Django **view**, and how does it process a request?
19. What is a Django **model**, and how does it represent database data?
20. What is Django ORM, and why is it useful?
21. What are Django migrations? Explain the difference between `makemigrations` and `migrate`.
22. How would you create a **CRUD operation** in Django?
23. What is the purpose of `settings.py` in a Django project?
24. How would you connect a Django application with **MySQL or PostgreSQL**?
25. What is the difference between Django **Templates and Views**?
26. How would you create a login/signup functionality in Django?

REST APIs & Web Integration

27. What is a RESTful API, and how would you implement one in a Django application?
28. Explain the difference between **GET, POST, PUT, PATCH, and DELETE** methods.
29. What is the difference between **GET and POST** from a backend perspective?
30. What is JSON, and why is it commonly used for communication between frontend and backend?
31. Explain HTTP status codes **200, 201, 400, 401, 403, and 404**.
32. How would you debug an API that is returning a **404 or 500 error**?
33. How would you connect a Django backend with a frontend built using **HTML, CSS, and JavaScript**?

SQL & Database

34. What is the difference between a **Primary Key and Foreign Key**?
35. Explain SQL `SELECT`, `INSERT`, `UPDATE`, and `DELETE` with examples.
36. What are SQL **JOINs**? Explain INNER JOIN and LEFT JOIN with an example.
37. How would you retrieve data from two related tables in SQL?
38. What is database normalization, and why is it important?

Git & Practical Development

39. Explain the Git workflow using `git clone`, `git add`, `git commit`, `git pull`, and `git push`.
40. Suppose you are working on a Django project and a page is not displaying the expected data from the database. **How would you systematically debug the issue?**

💡 3 Important Points

1. **Python + OOPs ko properly revise kar lena.** Sirf definition nahi, agar interviewer example ya code poochta hai toh confidently explain kar pao.
2. **Django ka complete flow ek baar project ke through revise karo — URL, View, Model, Database aur Response.** Apne project mein exactly kya kiya tha, ye especially clear hona chahiye.
3. **SQL, REST API aur Git ke basics bhi strong rakhna.** Interview mein practical situation dekar approach pooch sakte hain, isliye answer ratne ke bajay logic samajh ke jaana.

Best of luck for your interview. give it with calm mind. dont panic.